

Azure Platform Engineer Interview Prep

25-minute first-round video interview rehearsal guide

Role: Azure Platform Engineer | Remote US | Pacific Time support

1. Role Snapshot

Interview focus	What to show in your answers
Azure platform engineering	Hands-on experience supporting and modernizing Azure IaaS and PaaS platforms.
Terraform / IaC	Reusable modules, remote state, plan/apply process, peer review, and pipeline-based deployment.
Networking and security	VNets, subnets, NSGs, private endpoints, App Gateway, Front Door, Azure Firewall, WAF, RBAC, Key Vault, PIM.
Automation-first mindset	Azure DevOps pipelines, scripts, Log Analytics, Logic Apps, monitoring, reporting, and repeatable operations.
Production support	Troubleshooting, incident response, release readiness, monitoring, root-cause analysis, and postmortems.

Main message to repeat:

I am a hands-on Azure platform engineer with strong DevOps, Terraform, automation, Azure networking, security governance, monitoring, and production support experience.

2. 45-second Opening Answer

Use this when they ask: Tell me about yourself.

Rehearsal script:

My background is mainly in Azure cloud platform engineering, DevOps, infrastructure automation, and enterprise production support. In my recent role at WTW, I supported and modernized Azure-based platforms using Azure DevOps, Terraform/Bicep, App Services, AKS, Azure networking, monitoring, security governance, and release automation. I have worked on cloud migrations, CI/CD improvements, incident response, monitoring with App Insights and Log Analytics, and security reporting using tools like Wiz and Azure-native services. This role interests me because it is hands-on Azure engineering focused on Terraform, networking, security, and automation, which matches the work I have done around platform modernization, infrastructure governance, and reliable Azure operations.

3. Interview Strategy for a 25-minute Video Screen

- Keep most answers between 60 and 90 seconds.
- Use this simple format: Situation -> What I did -> Tool/service used -> Result.
- Be direct. Do not over-explain unless the interviewer asks for details.
- Repeat the themes they care about: Terraform, Azure networking, security, automation, CI/CD, and support.
- Confirm that Pacific Time hours work. Since you are in Central Time, 8 AM to 5 PM PT is 10 AM to 7 PM CT.

4. Key Talking Points to Use Naturally

Terraform / Infrastructure as Code

Say this:

I focus on repeatable infrastructure deployments using Terraform modules, variables, environment-specific tfvars, remote state, code review, and CI/CD approval gates. I prefer plan review before apply, and I treat infrastructure changes the same way as application code, with version control and peer review.

Azure Networking

Say this:

I have worked with Azure networking concepts such as VNets, subnets, NSGs, private access, load balancing, Application Gateway, App Services integration, and secure routing patterns. For enterprise environments, I usually think in terms of hub-and-spoke, controlled ingress/egress, least privilege, and observability.

Security and Governance

Say this:

My security approach is layered: NSGs for subnet and NIC-level filtering, Azure Firewall or centralized controls for traffic inspection, WAF for web-layer protection, RBAC/PIM for identity governance, Key Vault for secrets, and monitoring through Azure Monitor, Log Analytics, and security tooling.

Automation

Say this:

I like automation-first operations. I have used Azure DevOps pipelines, scripts, Logic Apps, Log Analytics queries, and reporting automation to reduce manual effort, improve visibility, and support production operations.

5. Likely Interview Questions and Strong Answers

Q1. How have you used Terraform with Azure?

I have used Terraform as part of Azure infrastructure automation to make deployments repeatable and easier to manage. My typical approach is to create reusable modules for common services such as resource groups, VNets, subnets, NSGs, App Services, storage, Key Vault, and monitoring resources. I keep variables and tfvars separate by environment, use remote state, and run terraform fmt, validate, plan, and apply through a pipeline. I also review the Terraform plan carefully before apply, especially for networking, firewall, and production changes, because a small rule or route change can impact application availability.

Q2. How would you design secure Azure networking for an enterprise app?

I would start with network segmentation. Usually I would use separate subnets for application, data, private endpoints, and shared services. I would apply NSGs with least-privilege inbound and outbound rules, use private endpoints where possible, and avoid exposing backend services directly to the internet. For internet-facing traffic, I would use Azure Front Door or Application Gateway depending on the use case. Front Door is better for global HTTP/HTTPS routing and edge delivery, while Application Gateway is commonly used for regional layer 7 routing inside a virtual network. For centralized inspection, I would use Azure Firewall with UDRs to control traffic flow.

Q3. What is the difference between Azure Front Door and Application Gateway?

Azure Front Door is a global entry point for HTTP/HTTPS applications. It is useful for global load balancing, acceleration, SSL offload, WAF at the edge, and routing users to the closest or healthiest backend. Application Gateway is regional and works inside Azure virtual networks. It is useful for layer 7 routing, path-based routing, SSL termination, and WAF protection closer to the application environment. In a real architecture, I might use Front Door at the edge and Application Gateway internally for regional routing and WAF, depending on the application and security requirements.

Q4. How do NSGs and Azure Firewall differ?

NSGs are used for basic traffic filtering at subnet or network interface level. They allow or deny traffic based on source, destination, port, and protocol. Azure Firewall is a centralized, managed, stateful firewall service that can inspect and control traffic more broadly across VNets, including north-south and east-west traffic. I would use NSGs for subnet-level segmentation and Azure Firewall for centralized egress, ingress, and inter-network traffic control.

Q5. How would you troubleshoot an App Service issue?

I would first confirm whether the issue is application, platform, networking, authentication, or dependency related. I would check App Service health, deployment history, configuration changes, application logs, App Insights, Log Analytics, and metrics such as CPU, memory, HTTP 5xx errors, response time, and dependency failures. If networking is involved, I would check private endpoint, VNet integration, DNS resolution, NSG rules, route tables, firewall logs, and backend connectivity. If it started after a deployment, I would review the release pipeline, app settings, connection strings, and recent infrastructure changes.

Q6. What is your experience with Azure SQL Managed Instance?

I have worked around Azure PaaS and database-connected application environments, including secure connectivity, monitoring, firewall rules, and application integration. For Azure SQL Managed Instance, I understand that it is a managed Azure SQL offering designed for strong SQL Server compatibility while giving the benefits of a managed PaaS platform. I would focus on private connectivity, backup and restore strategy, performance monitoring, identity integration, and secure access patterns.

Q7. What is your CI/CD approach for Terraform?

My preferred flow is pull request, code review, terraform fmt, terraform validate, security or static checks if available, terraform plan, approval, and then terraform apply. For production, I prefer separating plan and apply stages, storing the plan artifact when possible, and requiring approval before apply. I also like to protect state files, use remote backend storage, restrict who can run apply, and make sure service principals or managed identities follow least privilege.

Q8. How do you handle production incidents?

I follow a structured approach: confirm scope, check recent changes, review monitoring and logs, isolate the failing layer, communicate status, mitigate first, and then work on root cause. After recovery, I believe in documenting the timeline, root cause, corrective actions, and prevention steps. In my previous role, I was involved in production support, incident management, postmortems, release readiness, and monitoring improvements, so I am comfortable working in high-pressure enterprise environments.

Q9. What is your experience with Python and PowerShell?

My stronger scripting experience is around PowerShell, Azure CLI, YAML automation, and infrastructure automation. I have used scripting and AI-assisted workflows to support DevOps tasks, and I can read and modify Python when needed. I would describe PowerShell and Azure automation as my stronger day-to-day area, and I am comfortable improving my Python usage where the role requires it.

Q10. Why are you a good fit for this Azure Platform Engineer role?

I believe I am a strong fit because my background combines Azure platform engineering, DevOps automation, production support, security governance, and cloud modernization. I have worked in enterprise environments where reliability, change control, monitoring, and secure deployments are very important. This role matches the kind of hands-on Azure engineering work I have done, especially around infrastructure automation, networking, security, and operational support.

6. Personal Stories to Use**Story 1: Azure modernization**

At WTW, I was involved in Azure platform modernization and migration work for enterprise applications. The goal was to move toward more reliable, scalable, and easier-to-support Azure services. My role involved supporting cloud infrastructure, CI/CD, monitoring, release readiness, and production stability. This helped improve operational consistency and gave teams better visibility into deployments and incidents.

Story 2: Security and governance

I worked on security governance and reporting using Azure-native tools and Wiz. I helped automate visibility into vulnerabilities, cloud posture, and platform risks through reporting and operational follow-up. This aligns well with this role because Azure platform engineering is not just deployment. It also requires secure design, governance, monitoring, and continuous improvement.

Story 3: Observability and automation

I supported monitoring improvements using App Insights, Log Analytics, Azure Monitor, and automation. I have worked on reducing manual checks by creating reporting and alerting processes so teams could detect issues earlier and respond faster. This is important for a platform role because automation and observability directly improve reliability.

7. Questions to Ask the Interviewer

1. What are the main Azure modernization priorities for the first 90 days?
2. Is the Terraform environment already mature, or is part of the role to improve modules, pipelines, and standards?
3. How is Azure networking structured today: hub-and-spoke, centralized firewall, private endpoints, or a different model?
4. What are the biggest current pain points: deployment speed, security governance, networking, cost, or production support?
5. What CI/CD tools are currently used for Terraform deployments?

8. Final Closing Statement

Use this near the end:

I am very interested in this role because it is focused on hands-on Azure platform engineering, Terraform, networking, security, automation, and enterprise support. Those areas match my experience well. I am comfortable supporting Pacific Time hours, and I would be excited to contribute to Azure modernization, automation, and platform reliability for the team.

9. One-page Quick Rehearsal Sheet

Print this page separately or review it right before the video interview.

Topic	Short answer to remember
My fit	Azure platform engineering + Terraform + automation + networking/security + production support.
Terraform	Modules, variables, tfvars, remote state, validate, plan, approval, apply.
Networking	VNets, subnets, NSGs, private endpoints, App Gateway, Front Door, Azure Firewall.
Security	Least privilege, RBAC/PIM, Key Vault, WAF, private access, monitoring, governance.
Troubleshooting	Check scope, recent changes, logs, metrics, networking, dependencies, then mitigate and document RCA.
CI/CD	PR -> review -> validate -> plan -> approval -> apply.
Python	Stronger in PowerShell/Azure automation, comfortable reading and modifying Python as needed.
Schedule	Pacific Time hours work. 8 AM to 5 PM PT = 10 AM to 7 PM CT.

10. Practice Prompts

- Tell me about yourself.
- Walk me through your Terraform deployment process.
- Explain how you would secure an internet-facing Azure application.
- How would you troubleshoot a production outage in App Service?
- Describe your experience with Azure networking and security.
- Why are you interested in this contract role?

Reminder before recording:

Use a laptop, single screen, camera on, good internet, and do not switch networks during the interview. Pause briefly if the system lags before continuing your answer.